Arquitectura dirigida por eventos

Miguel Angel Moreno Ortiz
David Santiago Camacho Yate
Mariana Alejandra Martinez Duarte

Definicion

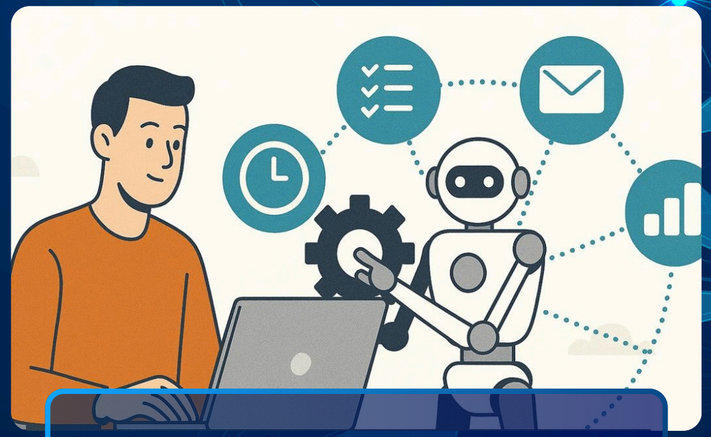
¿Que es ?

La arquitectura dirigida por eventos, también conocida como **Event-Driven Architecture (EDA)**, es un estilo de arquitectura de software en el que los diferentes componentes de un sistema se comunican mediante eventos, es decir, sucesos o cambios que ocurren dentro de una aplicación.

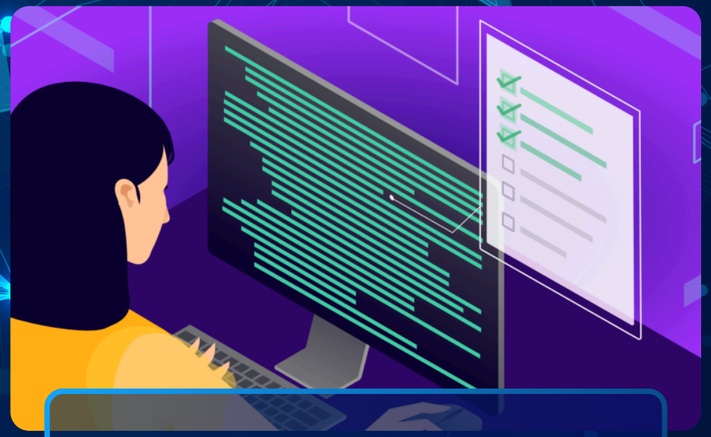
Un evento puede ser, por ejemplo, cuando un usuario inicia sesión, se registra una venta, llega un pedido o se agota un producto del inventario.

Definicion

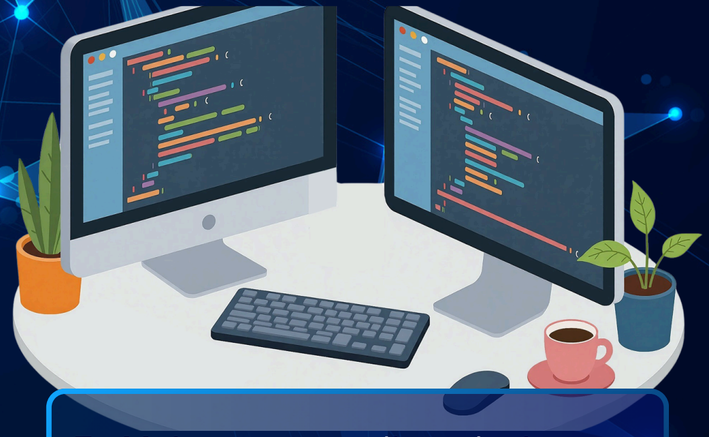
¿Para que sirve ?



1. Automatizar tareas: permite que el sistema realice acciones automáticamente cuando ocurre un evento, sin necesidad de hacer cada paso manualmente.



2. Facilitar la comunicación: permite que diferentes componentes del programa reaccionen a un mismo evento sin depender directamente unos de otros.



3. Mejorar y ampliar el sistema: facilita agregar nuevas funciones y procesar diferentes acciones de manera independiente.

¿Cómo funciona ?

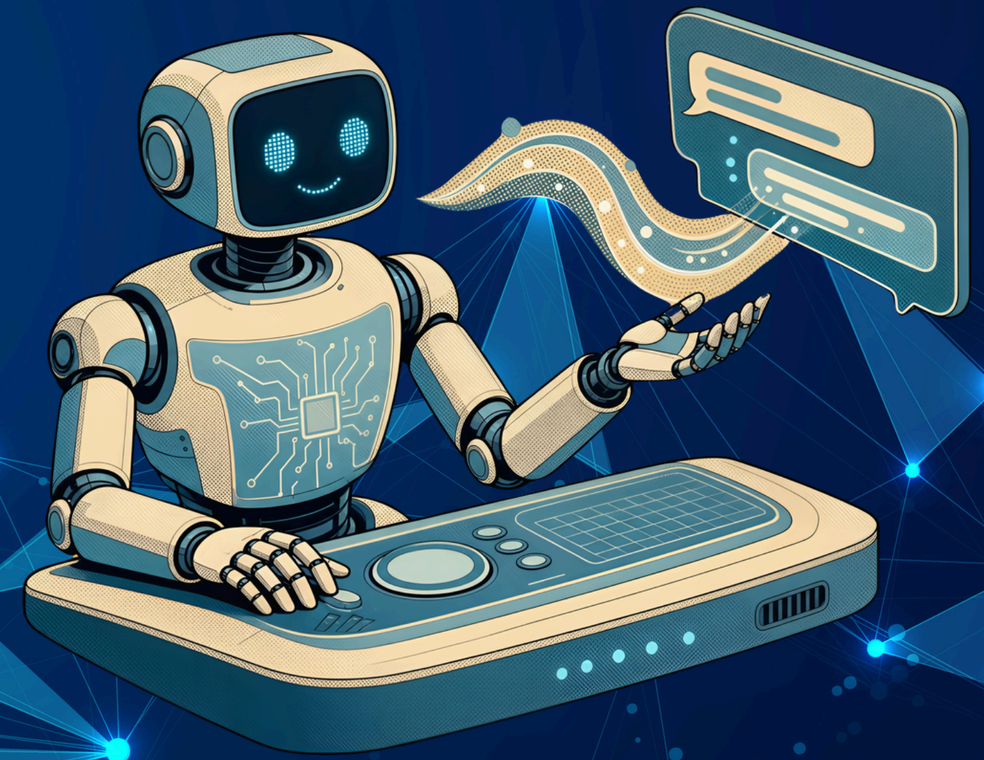
Funciona cuando ocurre una acción en el sistema y se genera un evento que permite a otros componentes reaccionar y ejecutar tareas automáticamente.

¿Cómo se organiza?

01 Productor: es el componente que genera un evento cuando ocurre una acción. Ejemplo: un cliente realiza una compra en una tienda virtual.

02 Intermediario (Broker): es el encargado de recibir el evento y enviarlo a los componentes interesados. Ejemplo: comunica que se ha registrado una nueva compra.

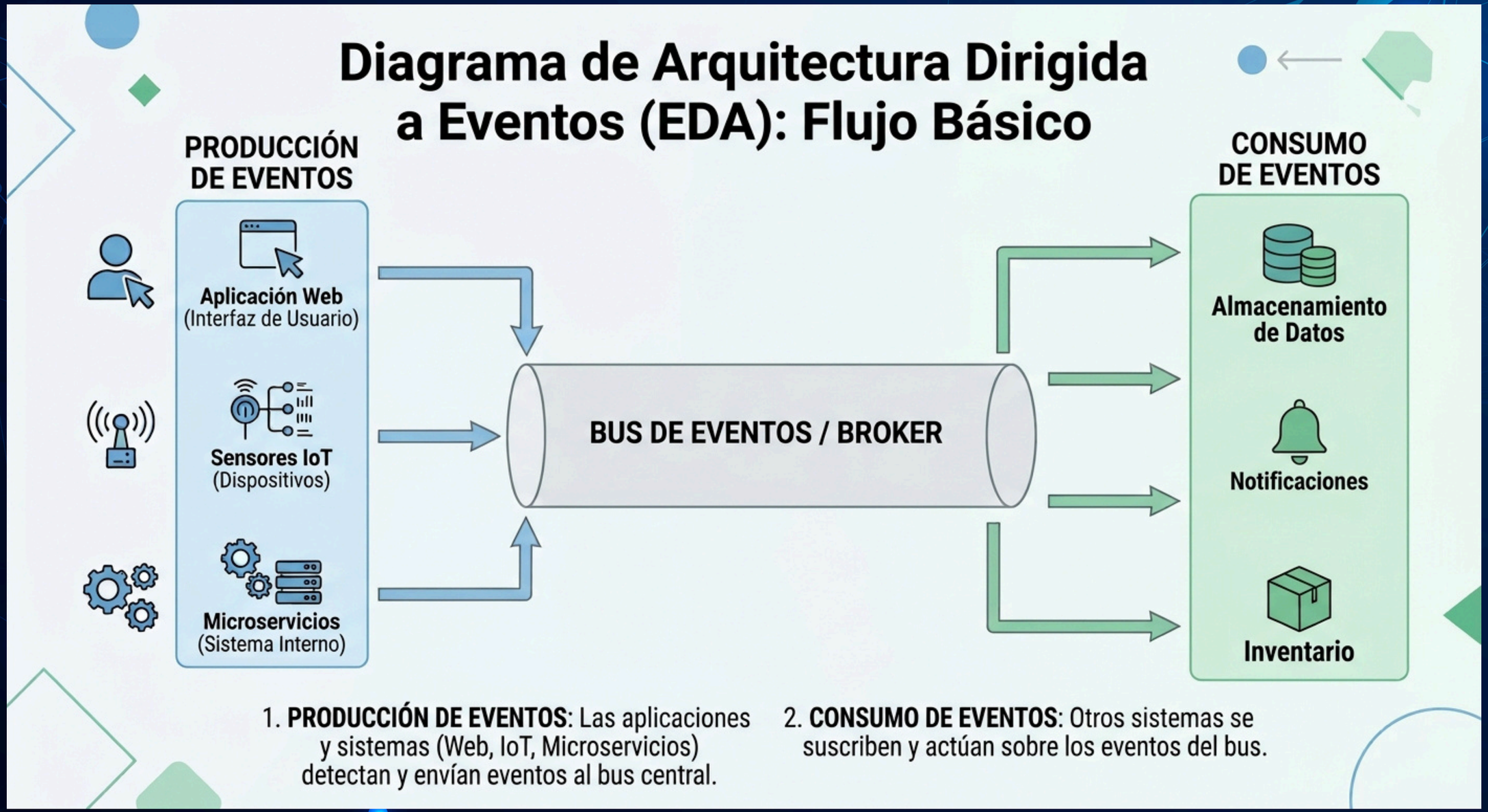
03 Consumidor: es el componente que recibe el evento y ejecuta una tarea relacionada. Ejemplo: un componente actualiza el inventario y otro envía la confirmación de compra al cliente.



Caracteristicas

- Desacoplamiento: los componentes pueden funcionar sin depender directamente unos de otros.
- Asincronía: las tareas pueden ejecutarse sin que el productor tenga que esperar a que todas terminen.
- Automatización: permite ejecutar acciones automáticamente cuando ocurre un evento.
- Escalabilidad: facilita agregar componentes para atender una mayor cantidad de eventos.
- Flexibilidad: permite incorporar nuevas funciones sin modificar todo el sistema.

Diagrama EDA



Ventajas

Desacoplamiento: los productores no necesitan conocer directamente a todos los consumidores.

Escalabilidad: puedes aumentar la capacidad de los consumidores que reciben más eventos.

Flexibilidad: puedes añadir nuevos consumidores sin modificar necesariamente al productor.

Procesamiento asíncrono: ciertas tareas pueden ejecutarse en segundo plano sin bloquear la solicitud principal.

Mejor distribución del trabajo: diferentes servicios pueden reaccionar al mismo evento de manera independiente.

Desventajas

- **Mayor complejidad:** hay que diseñar y gestionar los eventos, sus consumidores y sus posibles fallos.
- **Depuración más difícil:** una acción puede activar varios procesos distribuidos entre distintos componentes.
- **Consistencia de datos:** algunos cambios pueden tardar en reflejarse en otros módulos.
- **Duplicación de eventos:** un evento puede procesarse más de una vez si no se implementan controles adecuados.
- **Monitoreo más exigente:** es necesario rastrear los eventos, detectar errores y supervisar los tiempos de procesamiento.

Que tecnologias se desarrollan

01

Java

Es una excelente opción para desarrollar sistemas empresariales basados en eventos.

02

JavaScript

Permite construir aplicaciones dirigidas por eventos tanto en el servidor como en el navegador.

03

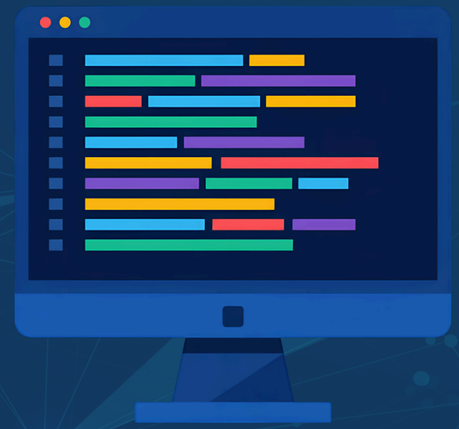
Python

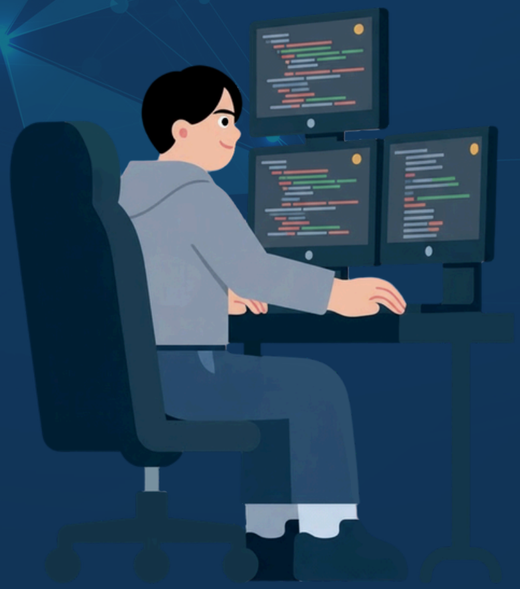
Destaca por su sencillez y por la variedad de bibliotecas disponibles.

04

C#

Cuenta con herramientas para crear sistemas distribuidos y aplicaciones empresariales.





2 Que tecnologias se desarrollan

05

Apache Kafka

Gestiona grandes volúmenes de eventos, permite conservarlos y volver a procesarlos. Es apropiado para sistemas de alto tráfico.

06

RabbitMQ

Recibe mensajes y los distribuye a las colas y consumidores correspondientes. Suele ser una opción accesible para aprender EDA

07

Redis Streams

Permite registrar eventos y distribuirlos a consumidores. Es útil cuando también necesitas aprovechar Redis para almacenamiento en memoria y baja latencia.

08

Amazon EventBridge y Azure Service Bus

Facilitan el enrutamiento de eventos y la comunicación entre servicios sin tener que administrar toda la infraestructura.

Conclusion

La arquitectura basada en eventos permite que los componentes del sistema trabajen de manera independiente, facilitando su adaptación y crecimiento. Sin embargo, requiere controlar posibles fallos, evitar el procesamiento repetido de mensajes y mantener la información sincronizada. Por ello, su implementación debe evaluarse según la complejidad y los requerimientos del proyecto.

Chain of Responsability

El patrón Chain of Responsibility (Cadena de Responsabilidad) es un patrón de diseño que permite enviar una petición a través de diferentes objetos, de manera que cada uno evalúa si está capacitado para atenderla o si debe delegarla al siguiente.

¿Para que sirve?

Sirve para distribuir la atención de una solicitud entre diferentes objetos, permitiendo que cada uno la procese si tiene la capacidad o la transfiera al siguiente responsable. Esto ayuda a reducir las dependencias entre clases, organizar mejor el código y facilitar la incorporación de nuevos manejadores sin modificar toda la estructura del sistema.

¿Como funciona?

Funciona creando una cadena de objetos conectados entre sí. Cuando llega una solicitud, el primer objeto la evalúa, si puede resolverla, la procesa, si no, la envía al siguiente objeto hasta encontrar quién pueda atenderla o llegar al final de la cadena.

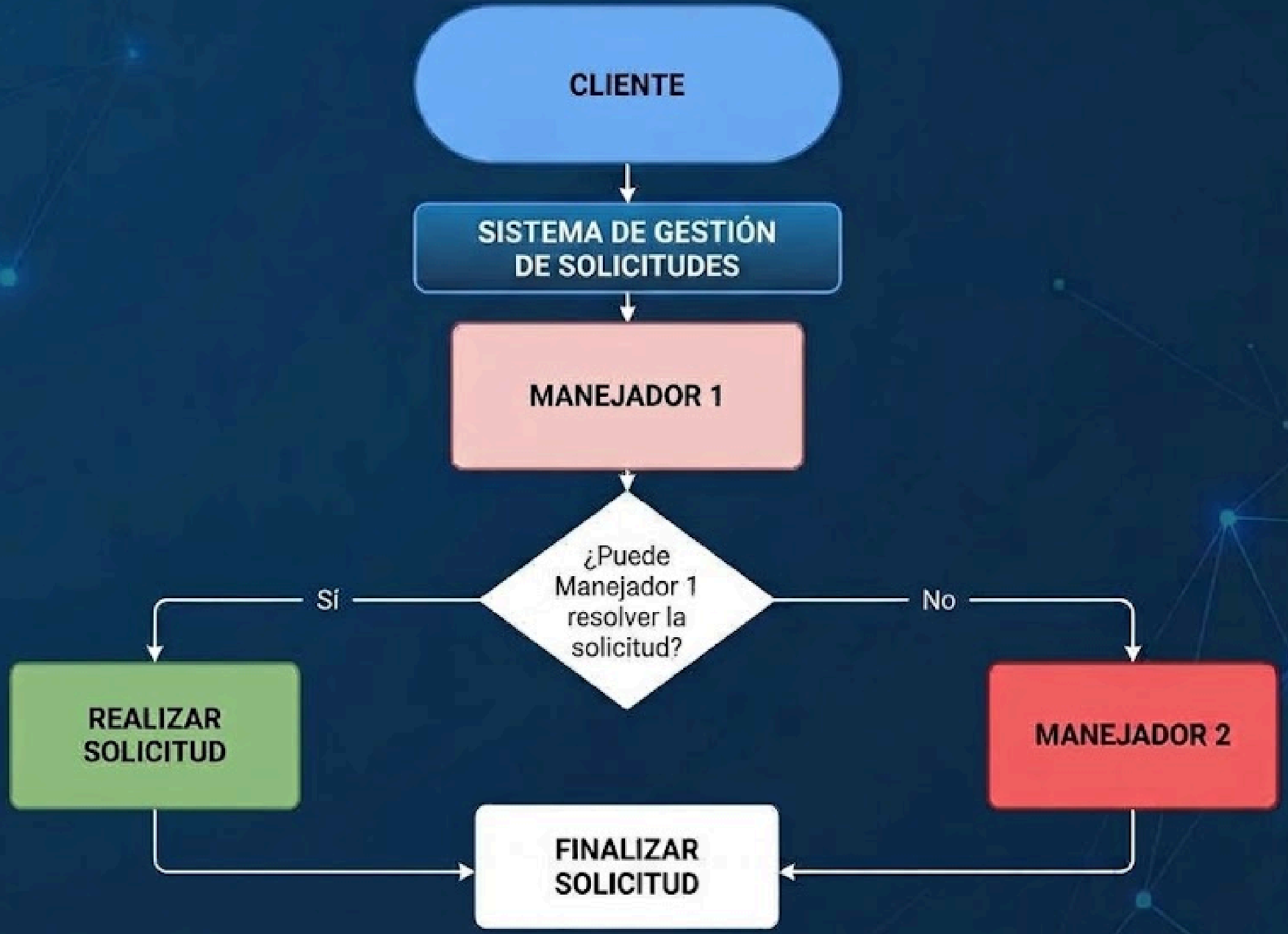
¿Cómo se organiza?

- **Manejador (Handler):** define la estructura común y establece cómo se transfiere una solicitud al siguiente objeto.
- **Manejadores concretos:** son las clases que implementan la lógica para atender solicitudes específicas.
- **Cliente:** crea o configura la cadena y envía la solicitud al primer manejador.

Caracteristicas

- **Desacoplamiento entre emisor y receptor:** El objeto que emite la solicitud no necesita conocer qué objeto exacto la va a resolver. Solo sabe que la solicitud será atendida por la cadena.
- **Delegación de responsabilidades:** cada objeto se encarga de atender las solicitudes que le corresponden.
- **Encadenamiento de objetos:** los manejadores están conectados y pueden transferir las solicitudes entre sí.
- **Flexibilidad:** permite agregar, quitar o cambiar manejadores sin modificar toda la estructura del sistema.
- **Procesamiento secuencial:** la solicitud pasa de un manejador a otro hasta que alguno la atiende o se termina la cadena.
- **Responsabilidad individual:** cada manejador contiene su propia lógica para evaluar y procesar las solicitudes.
- **Orden configurable:** los manejadores pueden organizarse en diferentes secuencias según las necesidades del sistema.

DIAGRAMA DE PROCESO DE SOLICITUD



Ventajas

- **Reduce el acoplamiento:** quien envía la solicitud no necesita conocer al objeto que la resolverá.
- **Facilita la ampliación:** permite agregar nuevos manejadores sin modificar toda la estructura del código.
- **Distribuye responsabilidades:** cada objeto se encarga de procesar las solicitudes que le corresponden.
- **Ofrece flexibilidad:** permite cambiar el orden de los manejadores según las necesidades del sistema.
- **Mejora el mantenimiento:** la lógica de cada tipo de solicitud se puede organizar en clases independientes.

Desventajas

- **Puede aumentar la complejidad:** una cadena extensa puede ser difícil de comprender y mantener.
- **No garantiza que se atienda la solicitud:** si ningún manejador puede procesarla, podría quedar sin respuesta.
- **Dificulta el seguimiento:** puede ser complicado identificar qué objeto terminó procesando una solicitud.
- **Puede afectar el rendimiento:** las solicitudes que recorren muchos manejadores requieren más procesamiento.
- **Requiere una configuración adecuada:** un orden incorrecto o una conexión mal establecida entre manejadores puede generar resultados inesperados.

¿Que tecnologias se desarrollan?

Se puede implementar con diferentes lenguajes de programación porque permite organizar una serie de objetos que reciben una solicitud y deciden si la procesan o la pasan al siguiente objeto de la cadena:

Java: Permite crear clases e interfaces para representar cada eslabón de la cadena. Facilita definir un método común para procesar solicitudes y delegarlas.

JavaScript: Permite crear funciones u objetos que procesan solicitudes de forma secuencial. Facilita encadenar funciones para aplicar diferentes validaciones.

Python: Permite crear clases o funciones que reciben solicitudes y las delegan a otros manejadores.

C#: Permite definir interfaces y clases para establecer una estructura común entre los manejadores. Facilita conectar los componentes mediante referencias al siguiente eslabón.

Command

Es un patrón de diseño de comportamiento que convierte una solicitud en un objeto independiente que contiene toda la información sobre la petición.

Para que sirve

– **Desacoplar sistemas:** Separa al objeto que emite la orden del objeto que sabe cómo ejecutarla

–**Gestionar colas y asincronía:** Permite almacenar solicitudes en colas de mensajes para procesarlas en segundo plano o en diferentes momentos.

Implementar sistemas de deshacer/rehacer: Como cada comando guarda el estado de la acción, es posible revertirla fácilmente

Como Funciona

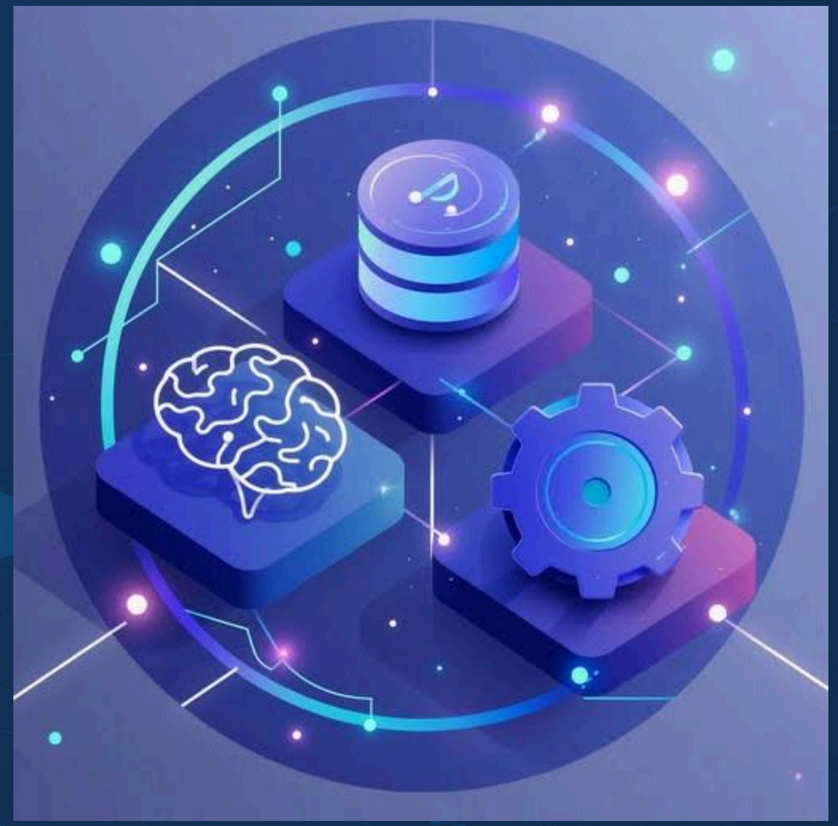
- El Cliente crea un comando concreto y le asigna un receptor específico.
- El cliente le entrega este comando al Invocador (por ejemplo, un botón o un menú).
- El usuario interactúa con el invocador, el cual llama al método **execute()** del comando.
- El comando toma la orden y llama al método correspondiente en el Receptor, que es quien realmente ejecuta la lógica de negocio.

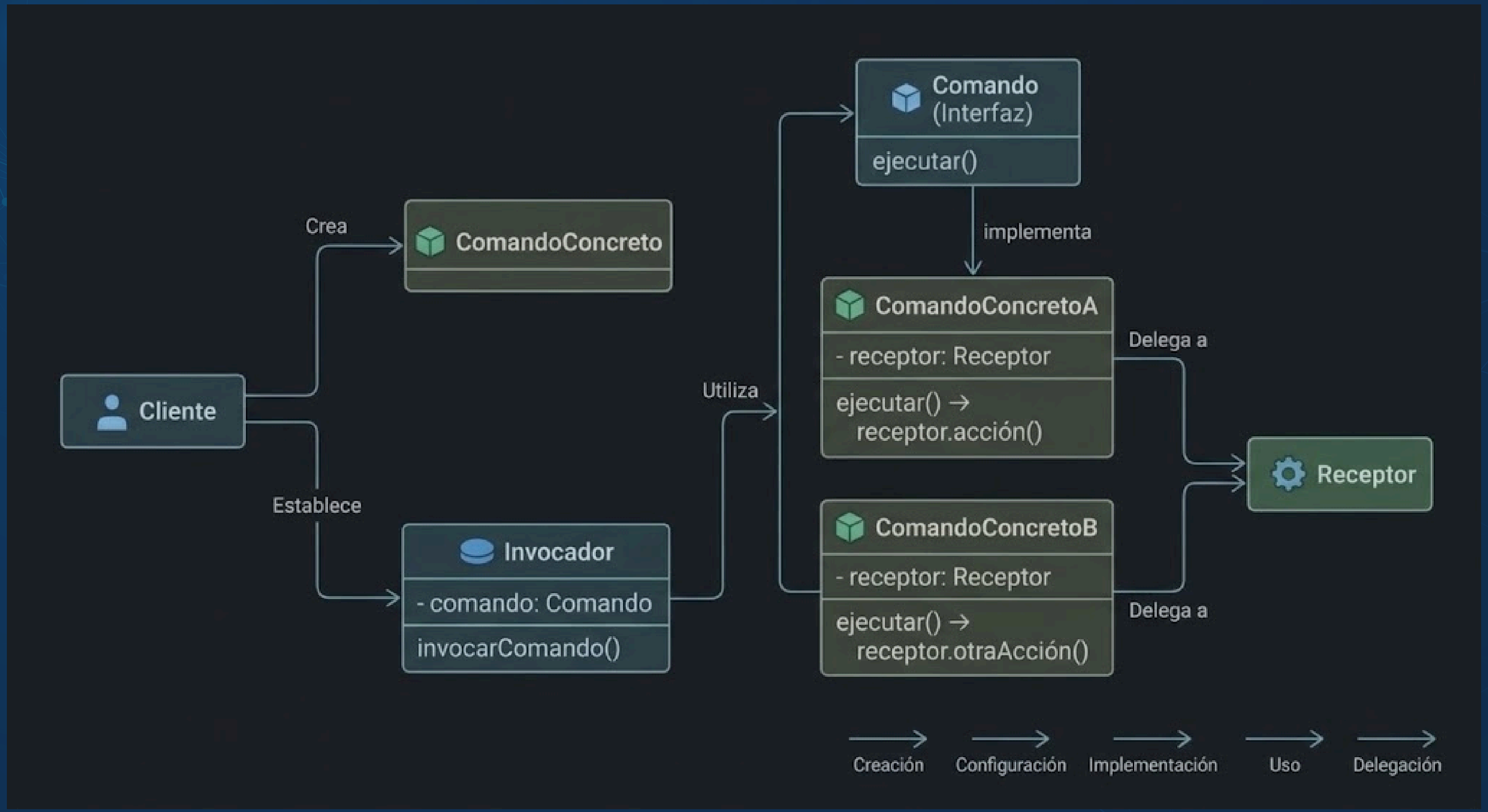
Características

Encapsulamiento: Cada comando encapsula una acción y sus parámetros específicos en una sola clase u objeto.

Interfaz Común: Todos los comandos implementan una interfaz genérica (generalmente con un método **execute()**).

Extensibilidad: Se pueden añadir nuevas operaciones o comandos sin necesidad de modificar el código existente (cumpliendo con el principio abierto/cerrado).




Comando
(Interfaz)
ejecutar()
implementa
Crea
ComandoConcreto
Cliente
Utiliza
ComandoConcretoA
- receptor: Receptor
ejecutar() →
receptor.acción()
Delega a
Receptor
Establece
Invocador
- comando: Comando
invocarComando()
ComandoConcretoB
- receptor: Receptor
ejecutar() →
receptor.otraAcción()
Delega a
Creación
Configuración
Implementación
Uso
Delegación

Ventajas

- **Desacoplamiento fuerte:** El emisor no necesita conocer la lógica interna ni los detalles del receptor de la orden.
- **Facilidad para auditoría:** Permite almacenar un historial de comandos ejecutados para revertirlos o registrarlos.
- **Composición de comandos**: Facilita la creación de macros o colas donde varios comandos se ejecutan de manera secuencial o en lote.

Desventajas

- **Proliferación de clases:** Puede incrementar significativamente el número de clases en el proyecto, ya que idealmente se crea una clase por cada comando.
- **Complejidad adicional:** Añade una capa extra de abstracción entre los objetos que puede ser innecesaria en aplicaciones pequeñas o sencillas.
- **Gestión de estado compleja:** Si los comandos modifican estados distribuidos o complejos, el mantenimiento del historial de reversión puede volverse difícil

Tecnologias

Javascript: Muy utilizado en interfaces de usuario y gestión de eventos (por ejemplo, botones o atajos de teclado). Se implementa mediante clases o incluso funciones que encapsulan la acción y permiten pasar callbacks directamente al invocador

C#: Es ampliamente adoptado junto con el patrón Mediator (usando librerías). Los controladores de una API envían un objeto "Command" que encapsula la solicitud de modificación de datos, desacoplando totalmente la capa web de la lógica de servicios.

Muchas gracias